

Controllo della Qualità del Servizio (QoS HTB) e Sicurezza tramite Dynamic ARP Inspection in Reti SDN basate su OpenFlow 1.3

RELAZIONE TECNICA DI PROJECT WORK

Studenti Referenti:

- Antonio Auricchio Matr. DE9000022
- Salvatore Marrandino Matr. DE9000118
- Alessandro Ortoli Matr. DE9000038

Repository e Consegna:

Repo Git: [giovannistanco/NCIS_unina \(Branch 2026\)](#)

Cartella PR: Auricchio_Marrandino_Ortoli

1 Introduzione e Obiettivi

Le architetture **Software-Defined Networking (SDN)** disaccoppiano formalmente il piano di controllo (*control plane*) dal piano dati (*data plane*), abilitando una logica di forwarding centralizzata, programmabile e flessibile. Questo project work affronta due problematiche critiche all'interno di infrastrutture SDN multi-subnet:

1. **Quality of Service (QoS):** Garantire la corretta allocazione di banda e l'isolamento dei flussi mission-critical rispetto a flussi concorrenti best-effort congestionanti su un canale condiviso (*bottleneck*).
2. **Sicurezza sul Piano Dati:** Rilevare e mitigare in tempo reale attacchi Man-in-the-Middle (MitM) basati su ARP Spoofing / Cache Poisoning, sfruttando l'intelligenza del controller OpenFlow senza richiedere agenti dedicati sugli host.

2 Ambiente Sperimentale e Prerequisiti Software

Il testbench è stato sviluppato e validato su ambiente Linux (**Ubuntu 26.04 LTS**). L'infrastruttura richiede l'integrazione di componenti software specifici per la gestione del piano dati, del piano di controllo OpenFlow e dei tool di generazione/ispezione del traffico:

- **Mininet:** framework di virtualizzazione di rete per emulare topologie su namespace Linux isolati.
- **Open vSwitch (OVS):** multilayer switch virtuale programmabile con supporto nativo OpenFlow 1.3 e database OVSDB.
- **Ryu SDN Framework:** framework di controllo in Python utilizzato per l'orchestrazione logica centralizzata e la gestione degli eventi OpenFlow.
- **iperf3:** tool diagnostico per la generazione e misurazione di throughput, jitter e packet loss per flussi UDP/TCP.
- **Scapy:** libreria avanzata di packet manipulation in Python, impiegata nel testbench per forgiare frame ARP malevoli a basso livello.

La predisposizione dell'ambiente e l'installazione delle relative dipendenze su sistema Debian/Ubuntu avviene tramite la seguente sequenza di comandi da terminale:

```
# Aggiornamento dell'indice dei pacchetti di sistema
sudo apt update && sudo apt upgrade -y

# Installazione dei componenti core, switch OVS e tool di rete
sudo apt install -y mininet openvswitch-switch openvswitch-testcontroller \
    iperf3 iproute2 python3-pip python3-scapy

# Installazione del controller Ryu e dipendenze compatibili
sudo pip3 install ryu eventlet==0.30.2

# Abilitazione e avvio del servizio Open vSwitch
```

```
sudo systemctl enable --now openvswitch-switch
```

3 Architettura della Rete e Topologia

La topologia emulata (topo.py) riproduce due domini broadcast indipendenti interconnessi a livello di rete da un router software Linux intermedio (r1):

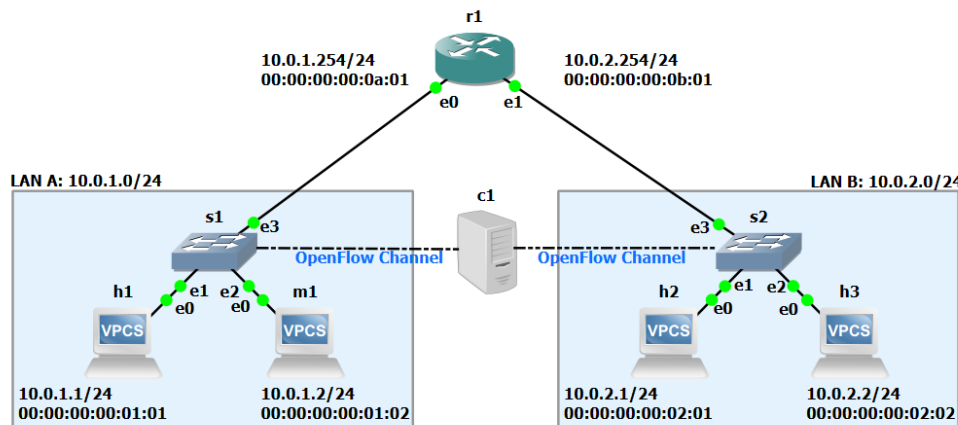


Figura 1: Architettura del testbench SDN: partizione delle subnet e canali OpenFlow verso C1.

- **LAN A (10.0.1.0/24):**
 - h1 (10.0.1.1, MAC 00:00:00:00:01:01): Industrial PC (IPC) / Workstation di Bordo (Macchina legittima).
 - m1 (10.0.1.2, MAC 00:00:00:00:01:02): terminale compromesso / attaccante interno.
 - s1: Open vSwitch collegato al gateway r1-eth0 (10.0.1.254) tramite la porta fisica 3 (s1-eth3).
- **LAN B (10.0.2.0/24):**
 - h2 (10.0.2.1, MAC 00:00:00:00:02:01): Postazione SCADA di Supervisione Real-Time (porta TCP/UDP 5201).
 - h3 (10.0.2.2, MAC 00:00:00:00:02:02): Data Historian / Archivio Diagnostico di Stabilimento (porta TCP/UDP 5202).
 - s2: Open vSwitch collegato all'interfaccia r1-eth1 (10.0.2.254).
- **Router L3 (r1):** Nodo Mininet specializzato basato su classe derivata LinuxRouter con `net.ipv4.ip_forward = 1`.

4 Ingegnierizzazione del Piano Dati e Configurazione QoS (HTB)

4.1 Scenario Applicativo e Caratterizzazione dei Flussi (Caso d'Uso)

Per ancorare la sperimentazione a uno scenario di fabbrica intelligente coerente con la topologia (costituita da postazioni workstation standard), abbiamo modellato l'host h1 come un **Industrial PC (IPC)**. Tale terminale, interfacciato con linee di produzione nella LAN A, genera intenzionalmente due flussi di traffico UDP verso i sistemi di supervisione situati nella LAN B:

- **Flusso Prioritario verso h2 (2 Mbps):** destinato alla trasmissione di **telemetria industriale critica in tempo reale** verso la postazione di controllo **SCADA** (h2). Trattandosi di misurazioni di processo e allarmi *delay-sensitive* e *loss-sensitive*, abbiamo imposto vincoli stringenti: banda garantita, oscillazioni di latenza trascurabili ($\text{jitter} < 2 \text{ ms}$) e zero perdite, la cui presenza invaliderebbe il monitoraggio dei macchinari.
- **Flusso Best-Effort verso h3 (12 Mbps):** flusso di **trasferimento massivo industriale (bulk data transfer)** diretto all'archivio storico di stabilimento (**Data Historian** h3), rappresentativo di dump diagnostici e sincronizzazioni di log. Tale flusso opera in modalità best-effort, tentando di occupare l'intero canale senza alcun controllo di congestione.

4.2 Razionale Architetture: Gestione del Traffico UDP tramite SDN

La scelta di impiegare flussi di trasporto **UDP** risponde a una specifica esigenza teorica e metodologica:

A differenza di TCP, che integra algoritmi end-to-end di controllo di congestione (quali Reno o Cubic) capaci di ridurre dinamicamente la congestione window a fronte di un packet loss, il protocollo UDP è privo di qualsiasi meccanismo di retroazione o controllo del ritmo di trasmissione.

In assenza di regolazione attiva di rete, l'immissione concorrente di due flussi UDP su un canale condiviso induce una competizione distruttiva: il bulk data transfer verso h3 (12 Mbps) satura l'intero buffer dell'interfaccia di uscita, provocando scarti non selettivi (*tail drop*) a carico anche della telemetria critica SCADA verso h2. L'assoluta cecità del protocollo di trasporto rende pertanto **indispensabile un'architettura di controllo centralizzata (SDN)**: il controller Ryu governa il piano dati di Open vSwitch, segregando i flussi a monte tramite flow rule puntuali su code hardware con scheduler **Hierarchical Token Bucket (HTB)**.

4.3 Configurazione delle Code HTB sul Bottleneck

La porta di uscita s1-eth3 costituisce il collo di bottiglia strutturale del sistema. Tramite Open vSwitch Database (**OVSDB**), sull'interfaccia è stato configurato uno scheduler HTB con capacità aggregata limitata a 10 Mbps:

- **Coda 0 (Telemetria Prioritaria per h2)**: CIR pari a 3 Mbps, con PIR estendibile fino a 10 Mbps.
- **Coda 1 (Bulk Transfer per h3)**: CIR pari a 7 Mbps, con estensione sulla banda residua fino a 10 Mbps.

4.4 Mitigazione del Bufferbloat tramite iproute2

Per impostazione predefinita, le implementazioni OVS associano alle classi HTB code FIFO con profondità elevata (1000 pacchetti), ritardando lo scarto da congestione e falsando il jitter. Nel progetto è stato applicato un limite rigido di buffer tramite tc:

```
tc qdisc change dev s1-eth3 parent 1:1 handle 10: pfifo limit 10
tc qdisc change dev s1-eth3 parent 1:2 handle 20: pfifo limit 10
```

La dimensione fissata a 10 pacchetti induce lo scarto deterministico dell'eccesso non appena il traffico supera la soglia di servizio, garantendo parametri di latenza stringenti.

5 Logica del Piano di Controllo SDN (controller.py)

L'applicazione Ryu opera secondo lo standard OpenFlow 1.3 ed è articolata in tre moduli cooperativi:

5.1 A. Routing Reattivo (L2 MAC Learning)

Allo switch features handshake, viene installata una regola di **Table-Miss** (priorità 0) che inoltra i frame non riconosciuti al controller (OFActionOutput(OFPP_CONTROLLER)). Quando un pacchetto giunge via Packet-In, il controller memorizza la mappatura (dpid, src_mac) → in_port e inietta una regola di flusso a priorità 10 per i successivi inoltri verso dst_mac.

5.2 B. Gestione Proattiva della Qualità del Servizio

All'attivazione dello switch d'ingresso s1 (dpid = 1), Ryu installa a priorità 50 le regole dedicate ai flussi UDP inter-subnet:

- Match: {eth_type = 0x0800, proto = UDP, dst_ip = 10.0.2.1} ⇒ **Actions:** {SetQueue(0), Output(3)}
- Match: {eth_type = 0x0800, proto = UDP, dst_ip = 10.0.2.2} ⇒ **Actions:** {SetQueue(1), Output(3)}

Questo meccanismo scavalca la regola di L2 learning standard e assicura l'instradamento nelle code hardware HTB dedicate.

5.3 C. Dynamic ARP Inspection (DAI) e Mitigazione degli Attacchi

- Una regola a priorità 1000 intercetta tutti i pacchetti con EtherType 0x0806 (ARP) e li forza a salire al controller.
- Ryu confronta i campi src_ip e src_mac del frame ARP con il dizionario autoritativo interno: trusted_arp[(dpid, in_port)].

- **Se viene rilevata un'incongruenza** (es. l'host `m1` che dichiara l'IP del gateway `10.0.1.254`):
 1. Viene generato un allarme di sicurezza formattato su console e nel file `stats.log`.
 2. Il pacchetto malevolo viene scartato immediatamente (nessuna operazione di `Packet-Out`).
 3. Viene iniettata sullo switch una regola di **DROP** hardware a priorità 100 associata alla porta e al MAC dell'attaccante (`actions = []`), isolando l'host compromesso dal resto della rete.

6 Risultati Sperimentali e Validazione

I test sono stati condotti in modalità automatica headless (`test_script.py` vs `test_noqos.py`).

6.1 Scenario 1: Verifica della QoS sotto Saturazione

È stato generato traffico UDP concorrente dall'IPC `h1` mediante `iperf3`: un flusso prioritario a 2 Mbps verso la postazione SCADA `h2` (porta 5201) ed un flusso best-effort saturante a 12 Mbps verso il Data Historian `h3` (porta 5202).

Tabella 1: Benchmark Prestazionale: Confronto Architetturale con e senza QoS

Parametro Prestazionale	Rete SENZA QoS (<code>test_noqos.py</code>)	Rete CON QoS HTB (<code>test_script.py</code>)
Throughput <code>h2</code> (SCADA) (2 Mbps nom.)	~ 1.4 Mbps (degradato)	2.0 Mbps (pieno regime)
Packet Loss <code>h2</code> (SCADA)	~ 51%	0.0% (nessuna perdita)
Jitter Flusso <code>h2</code> (SCADA)	> 15 ms	< 1.2 ms
Throughput <code>h3</code> (Historian) (12 Mbps inv.)	~ 8.4 Mbps	~ 8.0 Mbps
Packet Loss <code>h3</code> (Historian)	~ 30% (scarto non selettivo)	~ 33% (scarto isolato sull'eccesso)

Analisi: Senza QoS, l'eccesso di traffico generato da `h3` riempie l'unico buffer disponibile, scartando arbitrariamente oltre metà dei pacchetti del flusso prioritario `h2`. Con l'applicazione delle code HTB, Coda 0 isola interamente i 2 Mbps di telemetria di `h2` all'interno della riserva garantita di 3 Mbps. Il surplus residuo (1 Mbps) viene riallocato alla Coda 1, consentendo a `h3` di attestarsi esattamente sugli 8 Mbps massimi disponibili sul bottleneck senza interferire con il monitoraggio critico di `h2`.

6.2 Scenario 2: Validazione della Mitigazione ARP Spoofing

L'host attaccante `m1` ha eseguito lo script di attacco `test_arp.py` inviando frame ARP malevoli:

```
Ether(src="00:00:00:00:01:02")/ARP(op=2, psrc="10.0.1.254",
    hwsrc="00:00:00:00:01:02", pdst="10.0.1.1")
```

- **Esito su Ryu:** Il controller ha intercettato tempestivamente l'anomalia registrando:

```
SECURITY ALERT: ARP Spoofing Detected! DPID: 1, Port: 2,
Expected: 10.0.1.2/00:00:00:00:01:02, Got: 10.0.1.254/00:00:00:00:01:02
```

- **Esito sullo Switch:** Installata la flow entry di DROP hardware. Il successivo tentativo di comunicazione da parte di `m1` (`ping -c 2 10.0.1.1`) ha registrato un **100% packet loss**, dimostrando il blocco hardware e preservando l'integrità della cache ARP della workstation legittima `h1`.

7 Istruzioni per la Riproducibilità e Testbench Speculare

7.1 Metodologia di Benchmark e Testbench Speculare (No-QoS)

Per garantire un confronto prestazionale rigoroso a parità di condizioni al contorno, sono stati implementati moduli speculari a quelli di produzione:

- **Controller SDN Neutro** (`controller_noqos.py`): replica le logiche di L2 learning reattivo e i controlli di sicurezza DAI, omettendo solo l'installazione proattiva delle azioni `SetQueue` a priorità 50 verso le code HTB.

- **Topologia Dinamica in `test_noqos.py`:** per evitare di duplicare un file topologico statico (`topo_noqos.py`), lo script genera autonomamente a runtime una rete gemella priva di vincoli QoS sul bottleneck, evidenziando il reale impatto del traffico concorrente incontrollato.

7.2 Esecuzione dei Test

7.2.1 Scenario A: Test della Rete di Produzione (CON QoS HTB)

1. **Avvio Controller SDN (Terminale 1):** `ryu-manager controller.py`
2. **Esecuzione Automatica del Test (Terminale 2):**

```
sudo mn -c && mkdir -p results && sudo python3 test_script.py
```

3. **Verifica dei Risultati in `results/`:** `h2_iperf.txt` e `h3_iperf.txt` (metriche throughput/jitter/loss); `arp_attack.txt` e `ping_after_attack.txt` (esito attacco e blocco); `ovs_queue_stats.txt` (contatori code OVS).

7.2.2 Scenario B: Test del Benchmark di Controllo (SENZA QoS)

1. **Avvio Controller Neutro (Terminale 1):** `ryu-manager controller_noqos.py`
2. **Esecuzione del Test sulla Rete Speculare (Terminale 2):**

```
sudo mn -c && sudo python3 test_noqos.py
```

3. **Verifica dei Risultati in `results/`:** `noqos_h2_iperf.txt` e `noqosh3_iperf.txt` (metriche throughput/jitter/loss);